

REPUBLIQUE DU CAMEROUN

Paix-Travail-Patrie

SAINT JEAN UNIVERSITY INSTITUTE

INSTITUT UNIVERSITAIRE SAINT JEAN



REPUBLIC OF CAMEROON

***** *

Peace-Work-Fatherland

SAINT JEAN INGENIEUR

SAINT JEAN INGENIEUR



THEORIE DES SYSTEMES D'EXPLOITATION

REALISATION DE LA COMMANDE PS

OPTION : SYSTEME RESEAUX & TELECOMMUNICATION

INGE 3 SRT

Membres du groupe

- ONGUENE ANNE ALIMA
- SEUMO TCHATCHOUANG Bertrand
- TAMBO DJOUNTZO STEPHEN DARREN
- NGANKAK DZUKOU LOUIS-GEORGES
- FOTSO ERYANGE VERDIANE

Sous la supervision de :

Monsieur NGUIMBUS Emmanuel

Année académique :2025/2026

Table des matières

Introduction.....	2
I. Analyse de ps.....	3
1. Analyse du problème	3
Objectif de la commande ps :.....	3
A. Les paramètres sélectifs.....	4
B. Les paramètres de formatages.....	4
C. Les paramètres de visualisation et de hiérarchie.....	5
2. Analyse technique.....	5
A. Le fichier stat.....	6
B. Le fichier cmdline	7
C. Le fichier status	7
II. Conception	8
1. Algorithme du programme.....	8
a) Analyser les options de la ligne de commande (parse_args)	8
b) Ouvrir le dossier /proc	8
c) Itération et Filtrage des Entrées.....	8
d) Trier le tableau (par PID par défaut)	8
e) Pour chaque processus du tableau : afficher la ligne formatée.....	9
f) Fermer /proc et libérer les ressources	9
2. Structures de données et bibliothèques utilisées.....	9
3. Fonctions créées et utilisées	10
Conclusion	12

Introduction

Dans l'écosystème des systèmes d'exploitation de type UNIX, la gestion des processus constitue l'un des piliers fondamentaux de l'administration système. Parmi les outils essentiels mis à la disposition des utilisateurs et des administrateurs, la commande **ps** (Process Status) se distingue par sa capacité à fournir un instantané précis de l'activité du processeur et de la mémoire à un instant donné. Le présent projet s'inscrit dans le cadre de notre formation d'ingénieur, avec pour objectif la **conception et la réalisation d'une version simplifiée de la commande ps** en langage C. Ce défi technique ne se limite pas à un simple affichage textuel ; il nécessite une compréhension profonde de l'architecture du noyau Linux et de son système de fichiers virtuel **/proc**. À travers ce rapport, nous détaillerons notre démarche d'analyse et de conception, en mettant l'accent sur les points suivants : **L'exploration du système /proc** : Véritable interface entre l'espace noyau et l'espace utilisateur, permettant d'extraire les métadonnées des processus ; **L'architecture logicielle** : Comment nous avons structuré notre programme pour parser efficacement les fichiers **stat**, **status** et **cmdline**.

I. Analyse de ps

1. Analyse du problème

Objectif de la commande ps :

La commande **ps** (process status) est l'un des outils de surveillance les plus fondamentaux sous **Unix** et **Linux**. Son rôle principal est de fournir un instantané des processus actuellement en cours d'exécution sur le système. Son équivalent sous **windows** est **tasklist**. « **ps** » répond à des questions comme:

- Quels programmes tournent en ce moment sur ma machine ?
- Quel est le PID de mon serveur web ?
- Quelle est la consommation mémoire de chaque processus ?
- Depuis combien de temps ce processus est-il actif

```
(kali㉿kali)-[~]
└─$ ps
  PID TTY          TIME CMD
 11947 pts/3        00:00:00 zsh
 199335 pts/3        00:00:00 ps
```

1-Resultat de la commande ps

Fonctionnement interne de la commande ps :

Le noyau Linux maintient en mémoire une table des processus, qui est une structure de données contenant les informations de chaque processus actif. Un programme utilisateur comme **ps** peut lire les informations qu'il affiche qui sont dans le noyau grâce au dossier **/proc**.

/proc est un système de fichiers virtuel (il n'existe pas réellement sur le disque dur). Linux le crée en mémoire au démarrage et le met à jour en temps réel. Chaque fois que qu'un fichier est lu dans **/proc**, Linux vous donne les informations actuelles du

noyau. **/proc** permet donc seulement d'afficher les fichiers du noyau en temps réel.
Chaque dossier numérique de **/proc** correspond à un **PID**

Option de la commande **ps** :

La commande **ps** peut être utilisée avec plusieurs paramètres optionnels, notamment, les paramètres **sélectifs**, de **formatages**, paramètres de **visualisation**.

A. Les paramètres sélectifs

Ces paramètres filtrent la liste des processus que nous allons extraire de **/proc**.

Option	Utilisation	Ce qu'elle permet d'avoir
-A ou -e	ps -e	Tous les processus du système.
a (BSD)	ps a	Tous les processus avec un terminal (TTY), y compris ceux des autres utilisateurs.
x (BSD)	ps x	Tous les processus sans terminal (services, démons).
-u [user]	ps -u root	Processus appartenant à un utilisateur spécifique.
-p [pid]	ps -p 1	Informations sur un PID précis .
-C [nom]	ps -C bash	Processus correspondant à un nom de commande .
-G [gid]	ps -G 1000	Processus par ID de groupe .

B. Les paramètres de formatages

Ces paramètres modifient la structure du tableau de sortie.

Option	Utilisation	Ce qu'elle permet d'avoir
-f	ps -ef	Format complet : Affiche UID, PID, PPID, C, STIME, TTY, TIME, CMD.
-F	ps -eF	Format extra-complet : Ajoute la consommation mémoire (SZ, RSS) et le processeur (PSR).
u (BSD)	ps aux	Format utilisateur : Affiche %CPU, %MEM, VSZ, RSS, STAT. C'est le plus lisible pour l'humain.

-l	ps -el	Format long : Affiche les drapeaux (F), l'état (S) et la priorité (PRI, NI).
-j	ps -ej	Format job : Affiche le PGID (Process Group ID) et le SID (Session ID).

C. Les paramètres de visualisation et de hiérarchie

Ils permettent de regrouper les processus par **hiérarchie**, affiché l'**arbre généalogique** des processus avec des branches ASCII, et certains permettent d'affiche les informations de **sécurité SELinux**

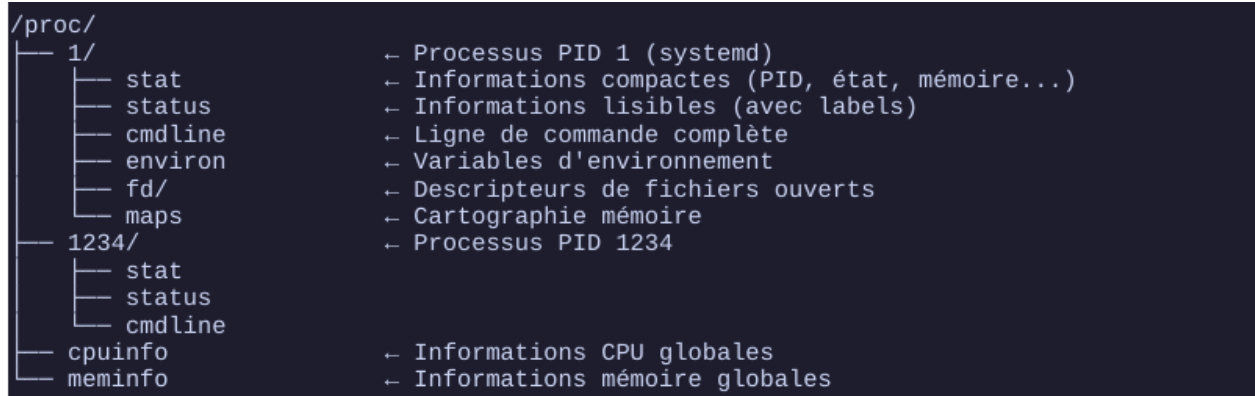
Option	Utilisation	Ce qu'elle permet d'avoir
f ou --forest	ps -e --forest	Affiche l' arbre généalogique des processus avec des branches ASCII.
-H	ps -eH	Regroupe les processus par hiérarchie (indentation).
-L	ps -eL	Affiche les Threads (LWP) au lieu de regrouper par processus.
-M	ps -eM	Affiche les informations de sécurité SELinux .

Il est également possible de combiner plusieurs paramètres différents. Nous pouvons avoir par exemple :

- **ps aux** : La vue "**Performance**". Tu as tout le système avec les détails CPU/RAM.
- **ps -ef** : La vue "**Système**". Tu as tout le système avec les liens Parent/Enfant (PPID).
- **ps -u USER** : La vue "**Personnelle**". Juste tes processus.

2. Analyse technique

La commande **ps** permet d'afficher le contenu de 3 fichiers à savoir : le fichier **stat**, le fichier **cmdline**, le fichier **status**.



2-Dossier /proc

A. Le fichier stat

C'est le fichier le plus compact. Il contient 52 champs d'informations système et toutes ces informations sont sur une seule ligne, séparées par des espaces. C'est le format idéal pour être analysé par un programme. Les champs sur les quels nous nous attarderons sont :

- **PID** : L'identifiant du processus.
- **Comm** : Le nom de l'exécutable entre parenthèses (ex: (bash)).
- **State** : Un seul caractère (R pour running, S pour sleeping, Z pour zombie).
- **PPID** : Le PID du processus parent.
- **TTY** : Le numéro de périphérique du terminal de contrôle.
- **utime** : Temps CPU passé en mode utilisateur (en "ticks" d'horloge).
- **stime** : Temps CPU passé en mode noyau.

```
# Exemple de contenu de /proc/1234/stat
$ cat /proc/1234/stat
1234 (bash) S 1000 1234 1234 34816 1235 4194304 500 0 0 0 10 5 0 0 20 0 1 0 12345
24576000 600 ...

# Décodage des champs (séparés par espaces) :
# 1234      → PID (identifiant du processus)
# (bash)    → Nom du processus (entre parenthèses)
# S         → État : R=Running, S=Sleeping, D=Disk wait, Z=Zombie, T=Stopped
# 1000      → PPID (PID du processus parent)
# ...et 47 autres champs !
```

3-Fichier stat

B. Le fichier cmdline

Alors que stat donne le nom abrégé du processus, cmdline contient la ligne de commande complète utilisée pour lancer le programme. Ici, les arguments sont séparés par des **caractères nuls (\0)** et non des espaces. La fin du fichier est marquée par un double \0. Concernant son usage dans **ps**, C'est ce qui s'affiche sous la colonne **COMMAND** ou **CMD**.

Cas particulier : Si le processus est un "**zombie**", ce fichier est vide.

```
# Afficher cmdline lisiblement (tr remplace \0 par espace)
$ cat /proc/1234/cmdline | tr '\0' ' '
bash

$ cat /proc/5678/cmdline | tr '\0' ' '
python3 /home/user/mon_script.py --verbose --output fichier.txt

# ATTENTION : Pour les processus kernel (threads du noyau),
# cmdline est VIDE. Ex: /proc/2/cmdline = vide
```

4-Fichier cmdline

C. Le fichier status

Ce fichier est conçu pour être lu par des humains. Il contient les mêmes informations que stat, mais sous forme de paires Clé : Valeur. Les informations sont écrites par ligne, et là nous avons une information par ligne.

```
$ cat /proc/1234/status
Name:   bash
Umask:  0022
State:  S (sleeping)
Tgid:   1234
Ngid:   0
Pid:    1234
PPid:   1000
TracerPid: 0
Uid:    1000 1000 1000 1000
Gid:    1000 1000 1000 1000
VmRSS:  4096 kB      ← Mémoire physique utilisée
VmSize: 24000 kB     ← Mémoire virtuelle totale
Threads: 1
```

5-Fichier status

II. Conception

1. Algorithme du programme

Notre programme est conçu sous la base de l'algorithme suivant :

a) Analyser les options de la ligne de commande (parse_args)

Le programme débute par l'analyse de la ligne de commande (avec la fonction **parse_args**). Ici, on vérifie si l'utilisateur a spécifié des options.

b) Ouvrir le dossier /proc

Le programme tente d'ouvrir le répertoire /proc pour récupérer les informations demandées.

c) Itération et Filtrage des Entrées

/proc contient plusieurs entrées (répertoires) représentant les processus en cours.

Pour chaque entrée dans /proc:

si l'entrée est un nombre (PID) :

lire les informations du processus (stat, status, cmdline)

si le processus correspond aux filtres (options) :

stocker ses infos dans un tableau

d) Trier le tableau (par PID par défaut)

Pour chaque PID identifié, le programme descend d'un niveau pour extraire les informations spécifiques :

- **Lecture de stat** : Extraction du PID, de l'état (Running, Sleeping, etc.) et du nom abrégé de la commande.
- **Lecture de status** : Récupération de l'UID (User ID) pour identifier le propriétaire du processus.
- **Lecture de cmdline** : Récupération de la ligne de commande complète et des arguments.
- **Conversion** : Les données brutes (comme l'UID numérique) sont converties en données lisibles (nom d'utilisateur via getpwnuid)
- **Lecture de stat** : Extraction du PID, de l'état (Running, Sleeping, etc.) et du nom abrégé de la commande.
- **Lecture de status** : Récupération de l'UID (User ID) pour identifier le propriétaire du processus.

- **Lecture de cmdline** : Récupération de la ligne de commande complète et des arguments.
- **Conversion** : Les données brutes (comme l'UID numérique) sont converties en données lisibles (nom d'utilisateur via getpwuid).

e) Pour chaque processus du tableau : afficher la ligne formatée

Les données sont formatées, elles sont stockées dans une structure de donnée et ensuite, le programme procède à l'affichage.

f) Fermer /proc et libérer les ressources

Il est primordial de fermer le répertoire **/proc** à la fin du programme.

2. Structures de données et bibliothèques utilisées

Pour notre programme, nous avons décidé de créer une structure **Processus** ayant pour champ de différents types :

- **Type int**: pid, ppid, uid, tty ;
- **Type char** : nom[256], etat, cmdline[512];
- **Type long** : rss ;
- **Type double** : cpu.

Concernant les bibliothèques utilisées, nous avons eu besoin de :

- **stdio.h** : Pour les fonctions d'entrée/sortie applicables à des fichiers (**fget**, **fopen**, **fclose**, **fprintf**, **fscanf**) ;
- **stdlib.h** : Utilisé pour les conversions et la gestion des ressources (**atoi**) ;
- **string.h** : Pour les fonctions applicables à des chaînes de caractères(**strncmp**, **strncat**, **strcpy**, **strlen**) ;
- **dirent.h** : Qui va nous permettre de parcourir le contenu des répertoires du système de fichier (**DIR** est une structure qui représente le répertoire ouvert ; **struct dirent** qui est une structure qui contient les informations sur un élément trouvé à l'intérieur du répertoire : nom du fichier, numéro d'inode, type de fichier ; **opendir** ; **readdir** ; **closedir** ; **rewinddir**) ;
- **unistd.h** : Nous allons utiliser cette bibliothèque uniquement pour gérer les options de la commande (**getopt**, **getuid**,**getgid**) ;
- **sys/types.h** : Permet de garantir que notre code est utilisable sur différents systèmes Unix (**pid_t**, **uid_t**, **gid_t**, **size_t**) ;

- **pwd.h** : Elle permet d'accéder à la base de données des utilisateurs du système (**struct passwd**, **getpwuid**, etc) ;
- **errno.h** : C'est un fichier d'en-tête de la bibliothèque standard.

3. Fonctions créées et utilisées

En plus des fonctions que nous fournis les bibliothèques, nous avons créé d'autres fonctions qui nous seront utiles à savoir :

- **est_un_nombre(const char *s)** : Cette fonction nous permet de vérifier si une chaîne de caractère ne contient que des chiffres. On l'utilisée pour distinguer les dossiers de PID ("1234") des fichiers système ("cpuinfo") dans /proc. Elle retourne 1 si la chaîne ne contient que des chiffres, 0 si non.
- **lire_stat(int pid, Processus *proc)** : Lit et analyse le fichier **/proc/[pid]/stat** pour extraire les informations principales d'un processus. C'est la fonction centrale de collecte de données. Elle remplit les champs suivants de la structure Processus : PID, PPID, état, mémoire **RSS**, **PGID**, **SID**, nom du processus, pourcentage CPU (calculé depuis **/proc/uptime**), et temps CPU total. Appelle aussi **lire_tty()** en interne.
- **lire_cmdline(int pid, char *buf, size_t taille)** : Lit la ligne de commande complète d'un processus depuis **/proc/[pid]/cmdline**. Remplit le champ **cmdline** avec les arguments du processus (séparés par des espaces). Si le fichier est vide, retourne [kernel thread] ; s'il est inaccessible, retourne [defunct].
- **lire_tty(int pid, char *buf, size_t taille)** : Cette fonction nous permet de lire le terminal associé à un processus depuis **/proc/[pid]/status** (ligne TTY:). Elle nous aide à remplir le champ TTY de la structure Processus. Si aucun terminal n'est trouvé (processus daemon ou noyau), retourne "?".
- **lire_contexte_z(int pid, char *buf, size_t taille)** : Lit le contexte de sécurité SELinux d'un processus depuis **/proc/[pid]/attr/current**. Utilisée uniquement quand l'option **-Z** est activée, pour afficher l'étiquette SELinux dans la colonne **LABEL**.
- **parse_args(int argc, char **argv, ...)** : Elle permet d'analyser les arguments de la ligne de commande et initialise les variables d'options. Parcourt argv sans utiliser getopt, supporte les options groupées (ex: -aux), les options longues (-

-sort=), et les options avec argument (-p, -t, -n, -C, -G). Retourne 1 en cas de succès, 0 en cas d'erreur.

- **lister_threads(int pid, int **tids)** : Liste tous les **TID** (Thread IDs) d'un processus en parcourant **/proc/[pid]/task/**. Utilisée avec l'option **-L** pour énumérer les threads d'un processus. Alloue dynamiquement un tableau d'entiers avec malloc/realloc et retourne le nombre de threads trouvés.
- **afficher_arbre(Processus *procs, int nb, int parent, int niveau)** : Affiche récursivement les processus sous forme d'arbre hiérarchique parent-enfant. Utilisée avec l'option **-H**. Pour chaque processus dont le **PPID** correspond au parent courant, elle indente la ligne et rappelle elle-même pour afficher les enfants de ce processus (récursivité).
- **main(int argc, char **argv)** : Point d'entrée du programme. Orchestre l'ensemble du traitement. Appelle **parse_args()** pour les options, parcourt /proc via opendir/readdir pour collecter tous les processus (ou threads si **-L**), applique les filtres via **doit_afficher()**, trie via **qsort()**, puis lance l'affichage via **afficher_arbre()** (si **-H**) ou **afficher_entete()** + **afficher_ligne()**. Gère aussi l'allocation dynamique du tableau de processus.

Conclusion

La réalisation de ce projet de recreation de la commande ps a constitué une étape charnière dans notre apprentissage de la programmation système, en nous forçant à passer d'une vision abstraite du système d'exploitation à une manipulation concrète de ses rouages internes. En exploitant le système de fichiers virtuel **/proc**, nous avons pu mettre en application le paradigme fondamental d'Unix selon lequel « tout est fichier », apprenant ainsi à extraire, filtrer et formater des métadonnées complexes issues directement du noyau. L'implémentation de cet outil a nécessité une maîtrise rigoureuse du langage C, notamment pour la gestion des flux de répertoires avec **<dirent.h>**, l'analyse fine des arguments via **getopt** et la traduction des identifiants numériques en noms d'utilisateurs grâce à la bibliothèque **<pwd.h>**. Au-delà de l'aspect purement algorithmique, ce travail nous a confrontés aux réalités de la robustesse logicielle : l'utilisation systématique de **errno.h** pour diagnostiquer les erreurs de permission ou les disparitions fugaces de processus a souligné l'importance de la résilience dans le développement d'outils d'administration. En somme, ce projet ne s'est pas limité à la simple production d'une sortie textuelle organisée ; il a consolidé notre capacité à interagir avec l'espace noyau, à gérer efficacement les ressources système et à concevoir une architecture logicielle modulaire capable d'évoluer vers des fonctionnalités plus avancées comme le monitoring en temps réel ou la gestion des liens de parenté entre processus.